Compiler Design

CHAPTER 3

Top-Down Parsers

Neso Academy

Compiler Design

Introduction to Parsers

Outcome

- ☆ Definition of Parser.
- ☆ Ways of generating Parse trees.
- ☆ Classification of Parsers.

Parser:

A parser is a program that generates a parse tree for the given string, if the string is generated from the underlying grammar.

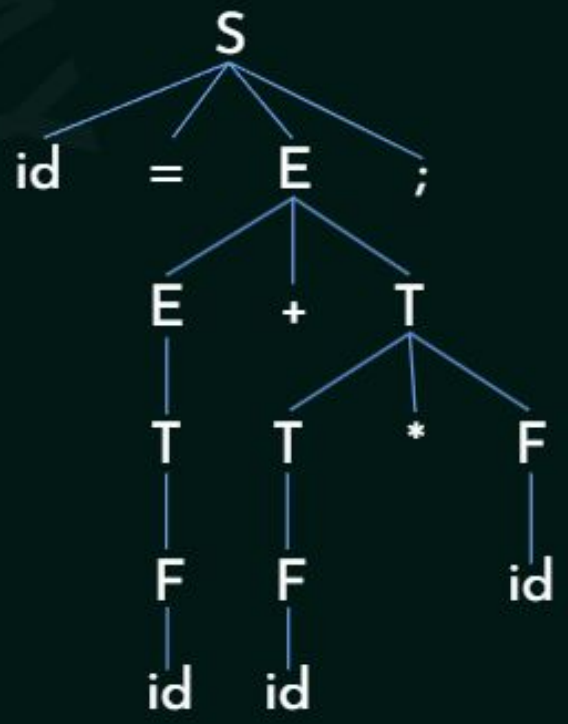
```
x = a+b*c;
```

Parser:

A parser is a program that generates a parse tree for the given string, if the string is generated from the underlying grammar.

```
id = id + id * id ;
```

$S \rightarrow id = E\ ;$
$E \rightarrow E + T | T$
$T \rightarrow T * F | F$
$F \rightarrow id$

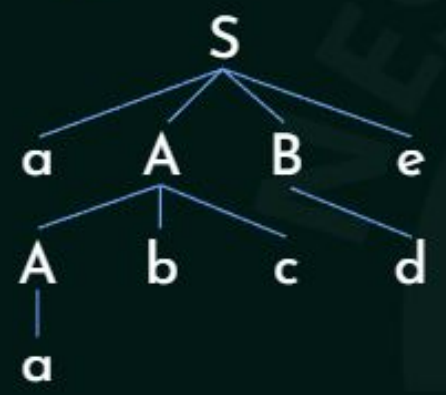


Generation of Parse Tree:

$S \rightarrow aABe,\ A \rightarrow Abc \mid a,\ B \rightarrow d$

aabcde

Top down approach:



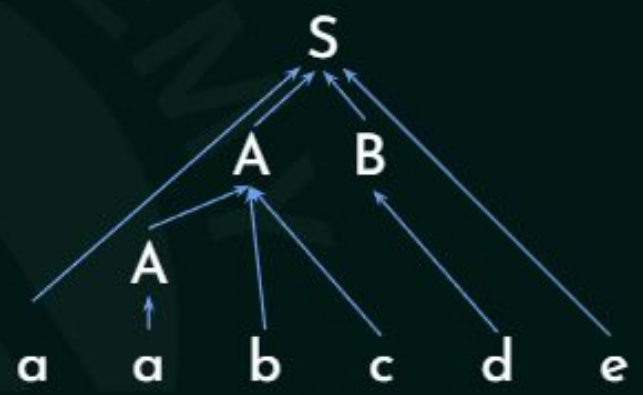
$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow aabcBe$
$\Rightarrow aabcde$

(Left most Derivation)

Decision:

Which production to use.

Bottom up approach:



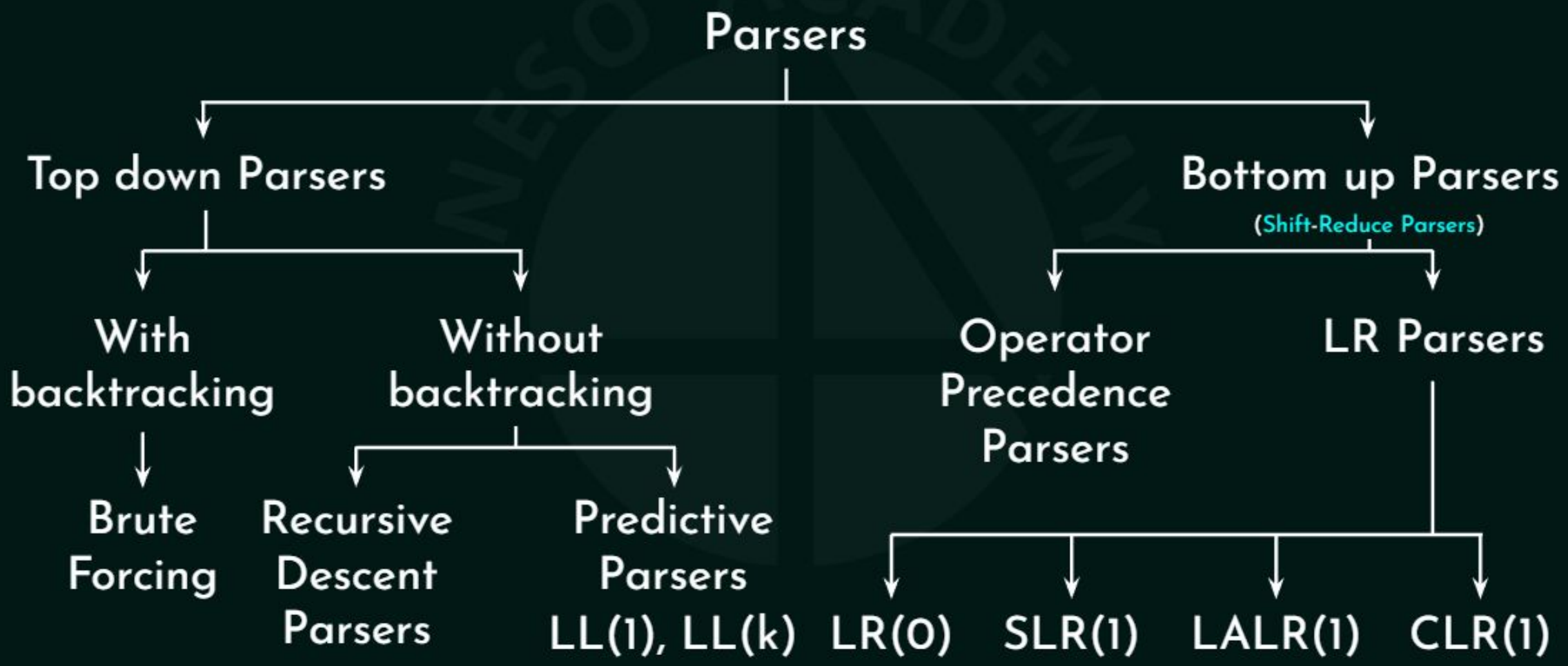
$S \Rightarrow aABe$
$\Rightarrow aAde$
$\Rightarrow aAbcde$
$\Rightarrow aabcde$

(Right most Derivation) - In reverse.

Decision:

When to reduce.

Classification of Parsers:



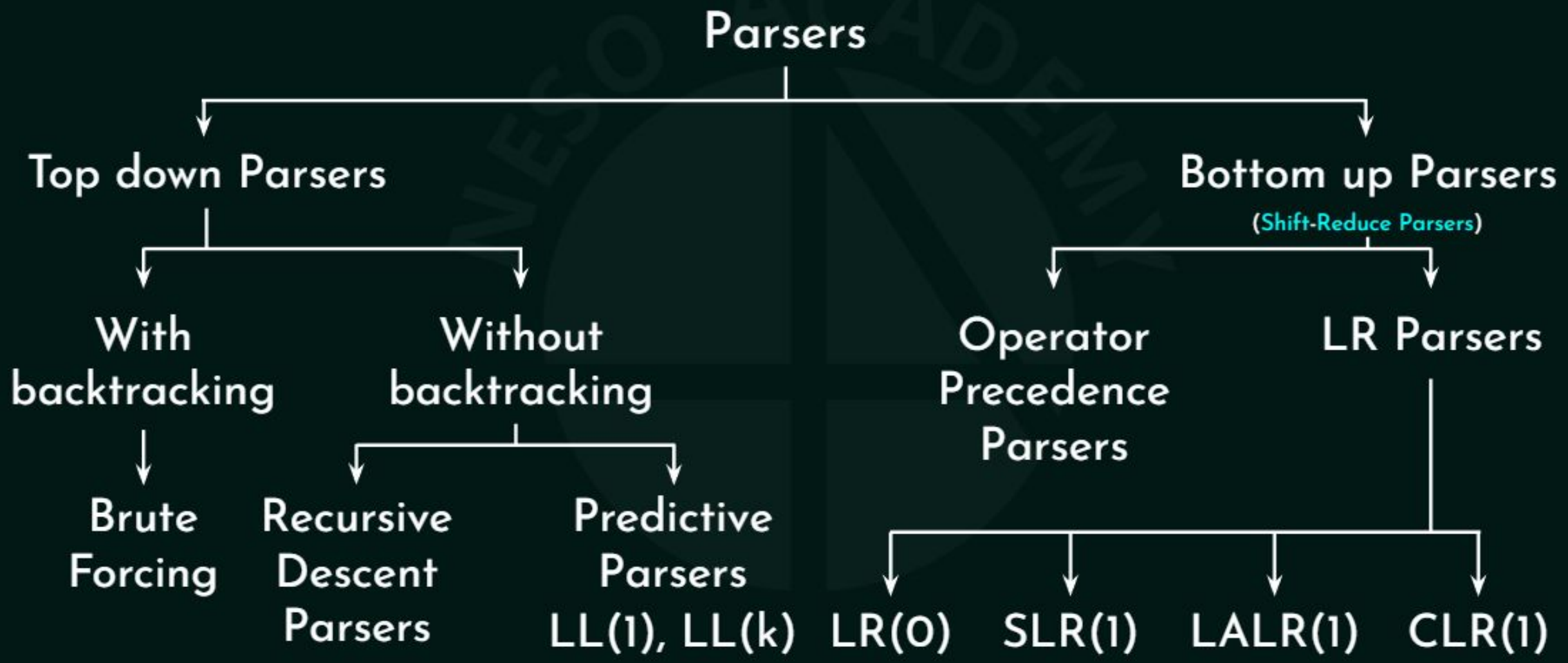
Compiler Design

Top down Parsers – Recursive Descent Parsers

Outcome

- ☆ Top down Parsers.
- ☆ Example of Recursive Descent Parser.

Classification of Parsers:

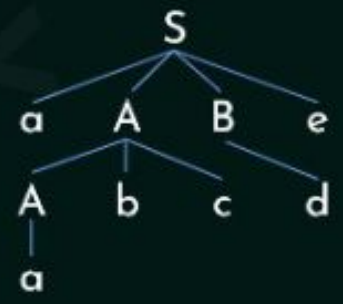


Top down Parser:

In order to construct Top down parsers the Context Free Grammars should not have,

1. Left Recursion,
2. Non-determinism.

Top down approach:



$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow aabcBe$
$\Rightarrow aabcde$

(Left most Derivation)

Decision:

Which production to use.

Recursive Descent Parser:

A recursive descent parser is a top-down parser built from a set of mutually recursive procedures (or a non-recursive equivalent) where each such procedure implements one of the non-terminals of the grammar. Thus the structure of the resulting program closely mirrors that of the grammar it recognizes.

Consider the following grammar having rules,

$$E \rightarrow iE'$$
$$E' \rightarrow +iE' \mid \varepsilon$$

Recursive Descent Parser:

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \varepsilon$

```
E()
{
    if(look_ahead=='i')
    {
        match('i');
        E'();
    }
}
```

```
E'()
{
    if(look_ahead=='+')
    {
        match('+');
        match('i');
        E'();
    }
    else
        return;
}
```

```
match(char c)
{
    if(look_ahead=c)
        look_ahead = getchar();
    else
        printf("ERROR!");
}
```

```
main()
{
    E();
    if(look_ahead=='$')
        printf("Parsing Successful!");
}
```

Recursive Descent Parser:

$E \rightarrow iE'$

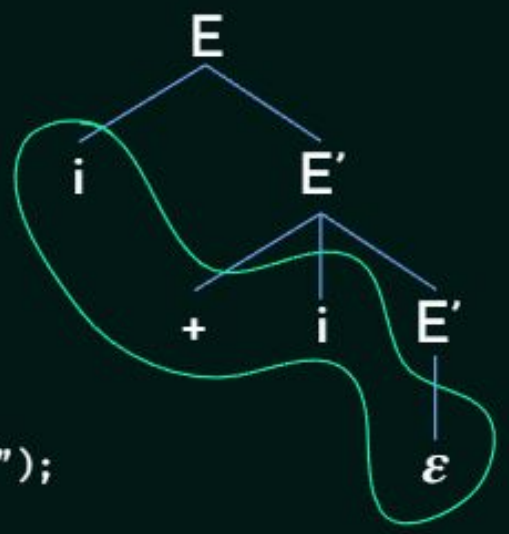
$E' \rightarrow +iE' \mid \varepsilon$

```
E()
{
        if(look_ahead='i')
        {
                match('i');
                E'();
        }
}
```

```
E'()
{
        if(look_ahead='+')
        {
                match('+');
                match('i');
                E'();
        }
        else
                return;
}
```

```
                i
match(char c)
{                i       i
        if(look_ahead=c)
                look_ahead = getchar();
        else
                printf("ERROR!");
}
```

```
main()
{
        E();
        if(look_ahead='$')
                printf("Parsing Successful!");
}
```



Input: i + i $

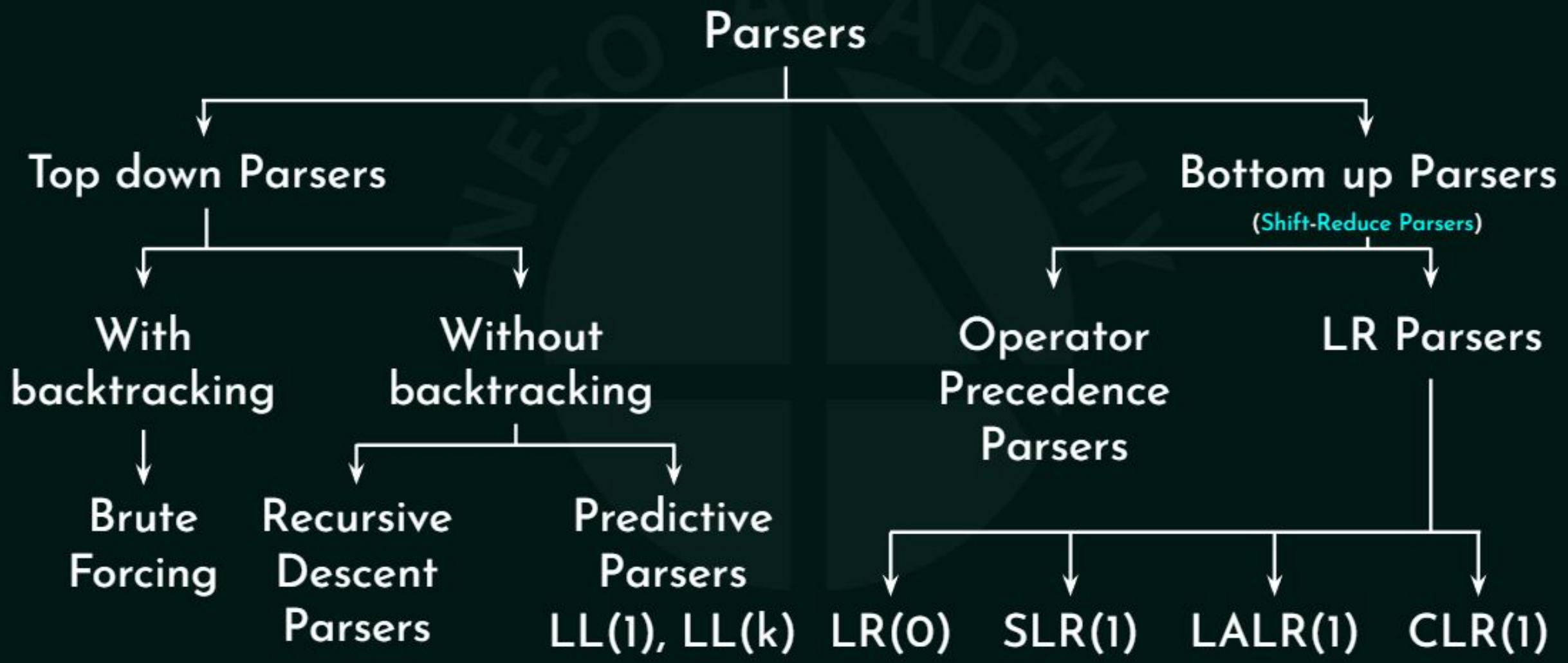
Compiler Design

Top down Parsers – LL(1) Parsers

Outcome

- ☆ Organization of LL(1) Parser.
- ☆ Understanding the concepts of FIRST & FOLLOW.

Classification of Parsers:



Recursive Descent Parser:

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \varepsilon$

```
E()
{
        if(look_ahead=='i')
        {
                match('i');
                E'();
        }
}
```

```
E'()
{
        if(look_ahead=='+')
        {
                match('+');
                match('i');
                E'();
        }
        else
                return;
```

```
match(char c)
{
        if(look_ahead=c)
                look_ahead = getchar();
        else
                printf("ERROR!");
}
```

```
main()
{
        E();
        if(look_ahead=='$')
                printf("Parsing Successful!");
}
```

Input: i + i $

Recursive Descent Parser:

$E \rightarrow iE'$

$E' \rightarrow +iE' \mid \varepsilon$

```
E()
{
        if(look_ahead=='i')
        {
                match('i');
                E'();
        }
}
```

```
match(char c)
{
        if(look_ahead=c)
                look_ahead = getchar();
        else
                printf("ERROR!");
}
```

```
E'()
{
        if(look_ahead=='+')
        {
                match('+');
                match('i');
                E'();
        }
        else
                return;
```

```
main()
{
        E();
        if(look_ahead=='$')
                printf("Parsing Successful!");
}
```

Input: i + i $

Recursive Descent Parser:

$E \rightarrow iE'$

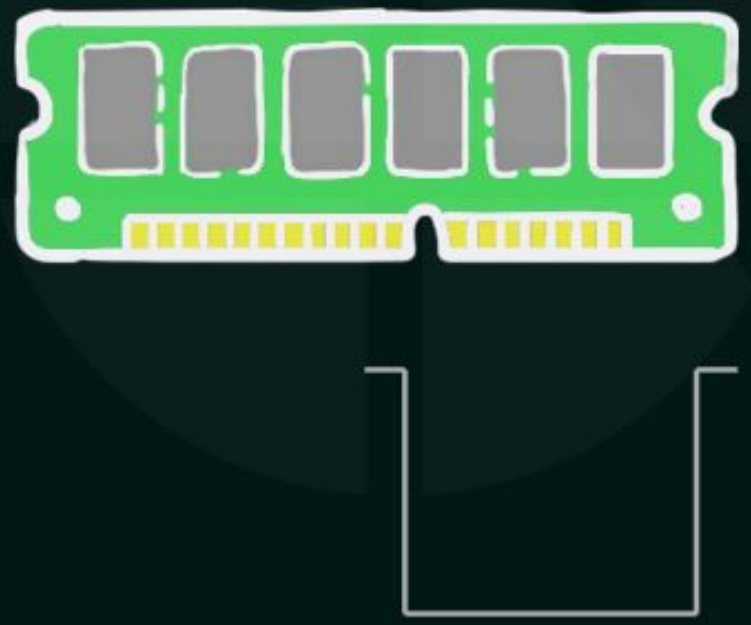
$E' \rightarrow +iE' \mid \varepsilon$

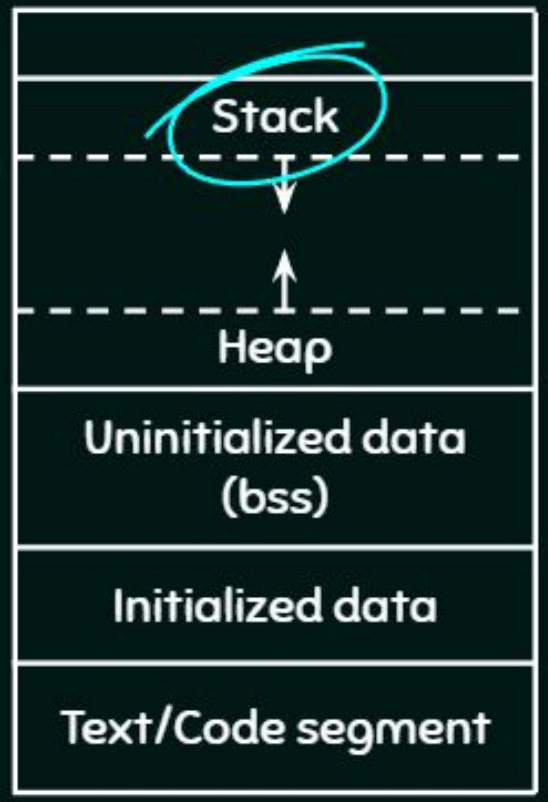
```
E()
{
    if(look_ahead=='i')
    {
        match('i');
        E'();
    }
}

E'()
{
    if(look_ahead=='+')
    {
        match('+');
        match('i');
        E'();
    }
    else
        return;
```

```
match(char c)
{
    if(look_ahead==c)
        look_ahead = getchar();
    else
        printf("ERROR!");
}

main()
{
    E();
    if(look_ahead=='$')
        printf("Parsing Successful!");
}
```





Input: i + i $

Recursive Descent Parser:

$E \rightarrow iE'$

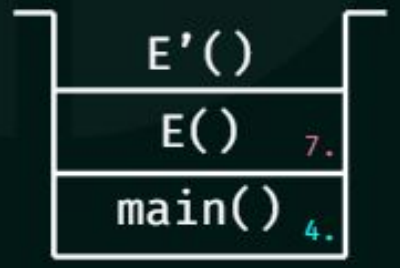
$E' \rightarrow +iE' \mid \varepsilon$

```
1.    E()
2.    {
3.          if(look_ahead=='i')
4.          {
5.                match('i');
6.                E'();
7.          }
8.    }
```

```
1.    E'()
2.    {
3.          if(look_ahead=='+')
4.          {
5.                match('+');
6.                match('i');
7.                E'();
8.          }
9.          else
10.               return;
```

```
1.    match(char c)
2.    {
3.          if(look_ahead=c)
4.                look_ahead = getchar();
5.          else
6.                printf("ERROR!");
7.    }
```

```
1.    main()
2.    {
3.          E();
4.          if(look_ahead=='$')
5.                printf("Parsing Successful!");
6.    }
```

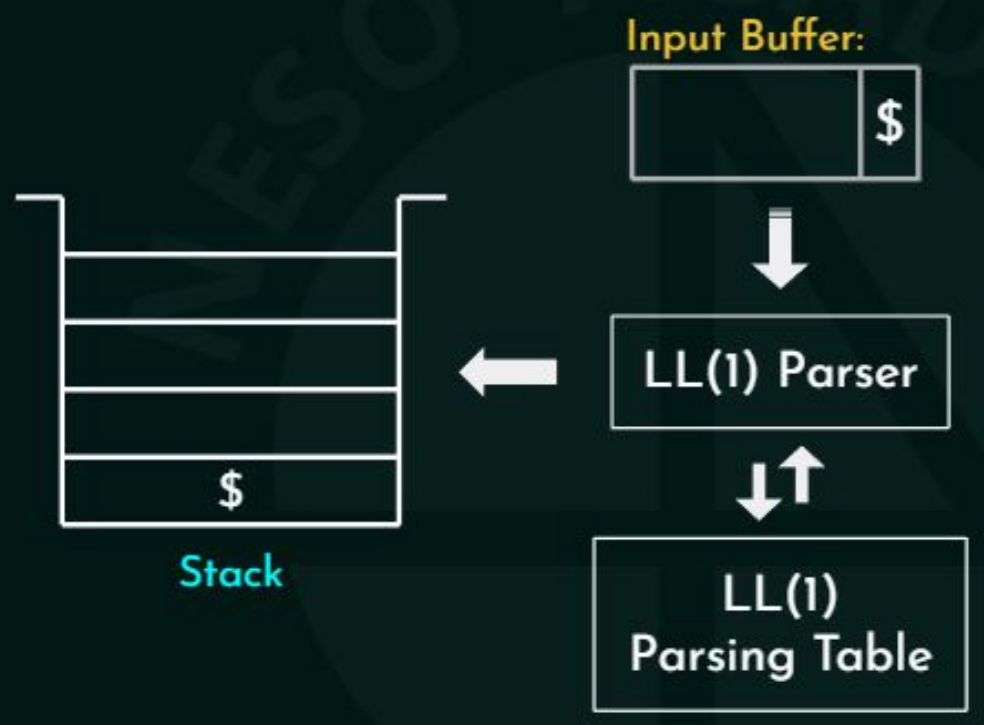


Input: i + i $

Classification of Parsers:

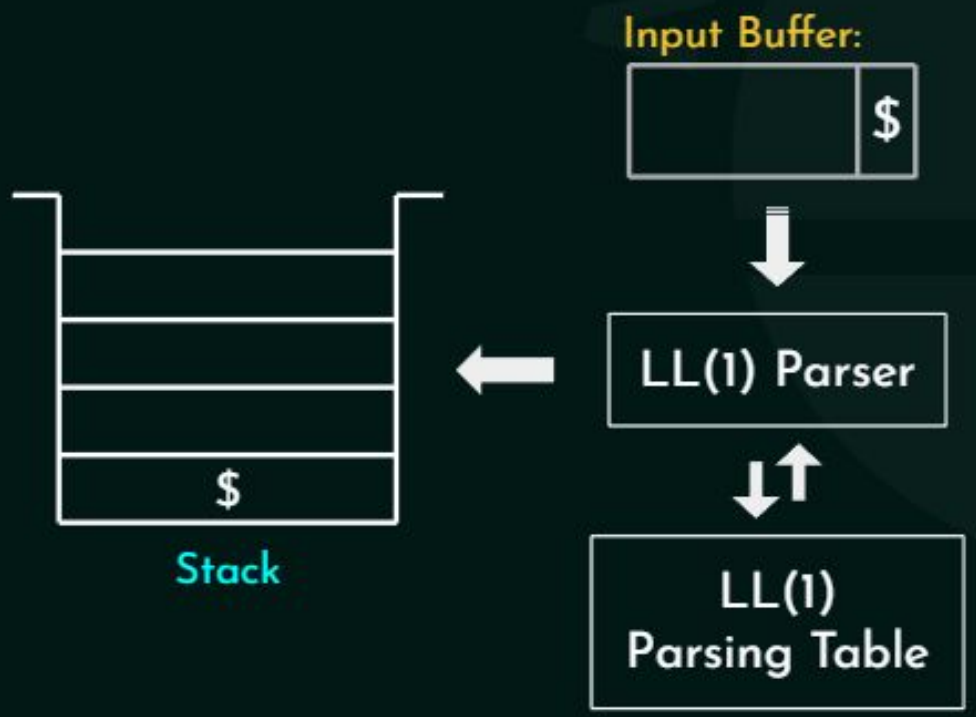
- Parsers
  - Top down Parsers
    - With backtracking
      - Brute Forcing
    - Without backtracking
      - Recursive Descent Parsers
      - Predictive Parsers LL(1), LL(k)
  - Bottom up Parsers (Shift-Reduce Parsers)
    - Operator Precedence Parsers
    - LR Parsers
      - LR(0)
      - SLR(1)
      - LALR(1)
      - CLR(1)

LL(1) Parser:

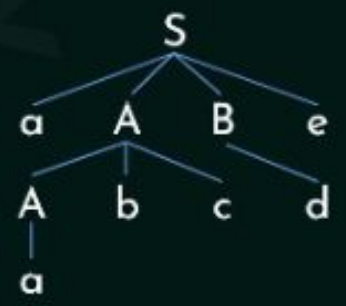


LL(1) Parser:

- "1" look-ahead symbol.
- Use Left most derivation.
- Scan from Left to right.



Top down approach:



$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow aabcBe$
$\Rightarrow aabcde$

(Left most Derivation)

Decision:
Which production to use.

LL(1) Parser:

1. FIRST():

Given any non-terminal of a CFG, if we derive all the possible strings from it, the first terminal(s) is the FIRST() of the non-terminal.

e.g.(1): $S \rightarrow aABC$
$A \rightarrow b$
$B \rightarrow c$
$C \rightarrow d$

FIRST(S):

FIRST(A):

FIRST(B):

FIRST(C):

LL(1) Parser:

1. FIRST():

   Given any non-terminal of a CFG, if we derive all the possible strings from it, the first terminal(s) is the FIRST() of the non-terminal.

e.g.(2): $S \rightarrow ABC$ FIRST(S): { a, b }

$A \rightarrow a \mid \varepsilon$

$B \rightarrow b$

$C \rightarrow c$

LL(1) Parser:

2. FOLLOW():

During the process of derivation, the terminal(s) that could follow the non-terminal are to be considered as FOLLOW() of the non-terminal.

e.g.: $S \rightarrow ABC$
$A \rightarrow a$
$B \rightarrow b \mid \varepsilon$
$C \rightarrow c$

FOLLOW(S): { $ }
FOLLOW(A): { b, c }
FOLLOW(B): { c }
FOLLOW(C): { $ }

Compiler Design

First & Follow

Outcome

- ☆ Step by step derivation of FIRST and FOLLOW.

Derivation of FIRST:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow id \mid (E)$

FIRST(F) = {id,(}

FIRST(T') = {$*$,ε}

FIRST(T) = {id,(}

FIRST(T) = FIRST(F) = { id, (}

Derivation of FIRST:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow id \mid (E)$

FIRST(F) = {id,(}

FIRST(T') = {$*$,ε}

FIRST(T) = {id,(}

FIRST(E') = {+,ε}

FIRST(E') = { +, ε }

Derivation of FIRST:

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow id \mid (E)$

FIRST(F) = {id,(}
FIRST(T') = {$*$,ε}
FIRST(T) = {id,(}
FIRST(E') = {+,ε}
FIRST(E) = {id,(}

FIRST(E) = FIRST(T) = { id, (}

Derivation of FIRST:

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow id \mid (E)$

	FIRST
$E \rightarrow TE'$	{id,(}
$E' \rightarrow +TE' \mid \varepsilon$	$\{+,\varepsilon\}$
$T \rightarrow FT'$	{id,(}
$T' \rightarrow *FT' \mid \varepsilon$	$\{*,\varepsilon\}$
$F \rightarrow id \mid (E)$	{id,(}

FIRST(F) = {id,(}
FIRST(T') = $\{*,\varepsilon\}$
FIRST(T) = {id,(}
FIRST(E') = $\{+,\varepsilon\}$
FIRST(E) = {id,(}

Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,ε}	{$,)}
$T \rightarrow FT'$	{id,(}	{+,$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*,ε}	
$F \rightarrow id \mid (E)$	{id,(}	

$$FOLLOW(T) = FIRST(E') = \{ +, \varepsilon \}$$

1. The following terminal symbol will be selected as FOLLOW.
2. The FIRST of the following non-terminal will be selected as FOLLOW.
3. If it is the right most in the RHS, the FOLLOW of the LHS will be selected.

Derivation of FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,ε}	{$,)}
$T \rightarrow FT'$	{id,(}	{+,$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*,ε}	{+,$,)}
$F \rightarrow id \mid (E)$	{id,(}	{*,+,$,)}

$$\text{FOLLOW}(F) = \text{FIRST}(T') = \{ * , \varepsilon \}$$
$$= \{*\} \cup \text{FOLLOW}(T) \cup \text{FOLLOW}(T')$$
$$= \{ *, +, \$,) \}$$

Derivation of FIRST & FOLLOW:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,ε}	{$,)}
$T \rightarrow FT'$	{id,(}	{+,$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*,ε}	{+,$,)}
$F \rightarrow id \mid (E)$	{id,(}	{*,+,$,)}

Compiler Design

First & Follow – Solved Problems (Set 1)

Outcome

- ☆ Three solved problems for determining First and Follow.

Q1: Consider the following grammar:

$$P \rightarrow xQRS$$
$$Q \rightarrow yz \mid z$$
$$R \rightarrow w \mid \varepsilon$$
$$S \rightarrow y$$

What is FOLLOW(Q)?

GATE 2017

(A) {R}

(B) {w}

(C) {w, y}

(D) {w, ε}

Q1: Consider the following grammar:

$P \rightarrow xQRS$

$Q \rightarrow yz \mid z$

$R \rightarrow w \mid \varepsilon$

$S \rightarrow y$

What is FOLLOW(Q)?

(A) {R}

(B) {w}

(C) {w, y}

(D) {w, ε}

FOLLOW(Q) = {w, y}

FIRST(R) = {w, ε}

FIRST(S) = {y}

Q2: Find the FIRST and FOLLOW of all the non-terminals:

$$S \rightarrow ABCDE$$
$$A \rightarrow a \mid \varepsilon$$
$$B \rightarrow b \mid \varepsilon$$
$$C \rightarrow c$$
$$D \rightarrow d \mid \varepsilon$$
$$E \rightarrow e \mid \varepsilon$$

Q2: Find the FIRST and FOLLOW of all the non-terminals:

	FIRST	FOLLOW
$S \rightarrow ABCDE$	{a, b, c}	{$}
$A \rightarrow a \mid \varepsilon$	{a, ε}	{b, c}
$B \rightarrow b \mid \varepsilon$	{b, ε}	{c}
$C \rightarrow c$	{c}	{d, e, $}
$D \rightarrow d \mid \varepsilon$	{d, ε}	{e, $}
$E \rightarrow e \mid \varepsilon$	{e, ε}	{$}

Q3: Find the FIRST and FOLLOW of all the non-terminals:

$$S \rightarrow Bb \mid Cd$$
$$B \rightarrow aB \mid \varepsilon$$
$$C \rightarrow cC \mid \varepsilon$$

Q3: Find the FIRST and FOLLOW of all the non-terminals:

	FIRST	FOLLOW
$S \rightarrow Bb \mid Cd$	{a, b, c, d}	{$}
$B \rightarrow aB \mid \varepsilon$	{a, ε}	{b}
$C \rightarrow cC \mid \varepsilon$	{c, ε}	{d}

Compiler Design

First & Follow – Solved Problems (Set 2)

Outcome

- ☆ Two solved problems for determining First and Follow.

Q1: Find the FIRST and FOLLOW of all the non-terminals:

$$S \rightarrow aBDh$$
$$B \rightarrow cC$$
$$C \rightarrow bC \mid \varepsilon$$
$$D \rightarrow EF$$
$$E \rightarrow g \mid \varepsilon$$
$$F \rightarrow f \mid \varepsilon$$

Q1: Find the FIRST and FOLLOW of all the non-terminals:

	FIRST	FOLLOW
$S \rightarrow aBDh$	{a}	{$}
$B \rightarrow cC$	{c}	{g, f, h}
$C \rightarrow bC \mid \varepsilon$	{b, ε}	{g, f, h}
$D \rightarrow EF$	{g, f, ε}	{h}
$E \rightarrow g \mid \varepsilon$	{g, ε}	{f, h}
$F \rightarrow f \mid \varepsilon$	{f, ε}	{h}

Q2: Find the FIRST and FOLLOW of all the non-terminals:

$$S \rightarrow ACB \mid CbB \mid Ba$$
$$A \rightarrow da \mid BC$$
$$B \rightarrow g \mid \varepsilon$$
$$C \rightarrow h \mid \varepsilon$$

Q2: Find the FIRST and FOLLOW of all the non-terminals:

	FIRST	FOLLOW
$S \rightarrow ACB \mid CbB \mid Ba$	{d, g, h, b, a, ε}	{$}
$A \rightarrow da \mid BC$	{d, g, h, ε}	{h, g, $}
$B \rightarrow g \mid \varepsilon$	{g, ε}	{$, a, h, g}
$C \rightarrow h \mid \varepsilon$	{h, ε}	{g, $, b, h}

Compiler Design

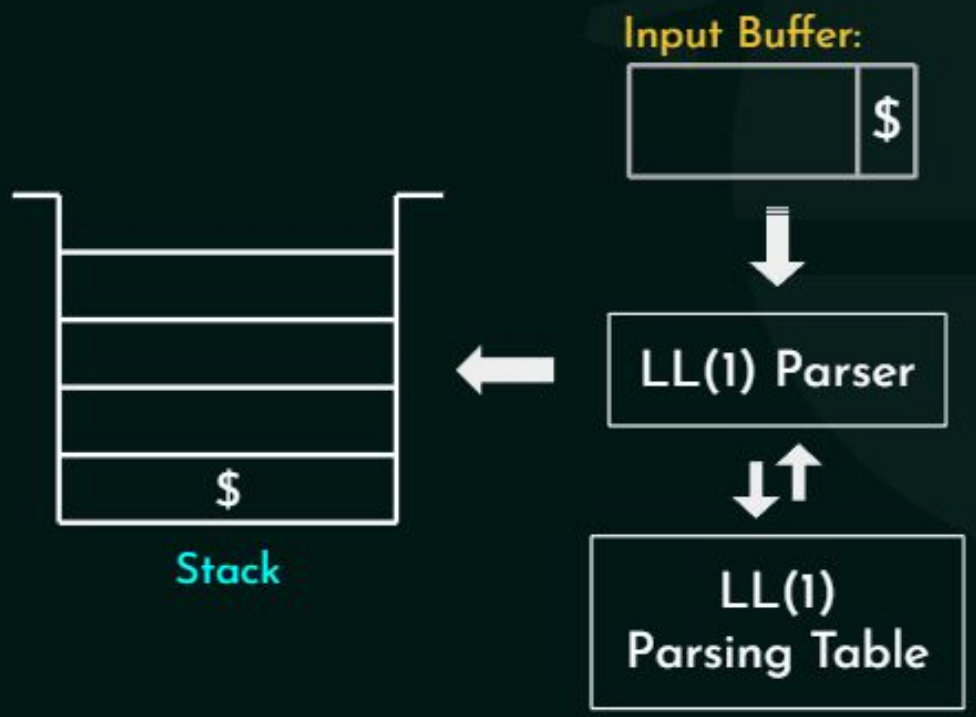
LL(1) Parsing Table

Outcome

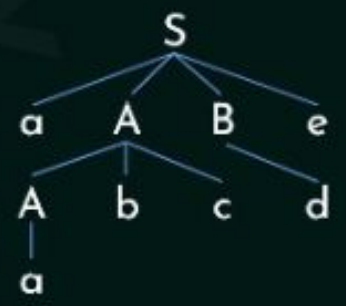
- ☆ Construction of LL(1) Parsing table.

LL(1) Parser:

- "1" look-ahead symbol.
- Use Left most derivation.
- Scan from Left to right.



Top down approach:



$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow aabcBe$
$\Rightarrow aabcde$

(Left most Derivation)

Decision:
Which production to use.

Construction of LL(1) Parsing table:

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,ε}	{$,)}
$T \rightarrow FT'$	{id,(}	{+,$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*,ε}	{+,$,)}
$F \rightarrow id \mid (E)$	{id,(}	{*,+,$,)}

Construction of LL(1) Parsing table:

	id	(	)	*	+	$
E	$E \rightarrow TE'$	$E \rightarrow TE'$				
E'			$E' \rightarrow \varepsilon$		$E' \rightarrow +TE'$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$				
T'			$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$	$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$	$F \rightarrow (E)$				

Rules:

1. All the ε-productions are placed under FOLLOW sets.
2. Remaining productions are placed under the FIRSTs.

	FIRST	FOLLOW
$E \rightarrow TE'$	{id,(}	{$,)}
$E' \rightarrow +TE' \mid \varepsilon$	{+,ε}	{$,)}
$T \rightarrow FT'$	{id,(}	{+,$,)}
$T' \rightarrow *FT' \mid \varepsilon$	{*,ε}	{+,$,)}
$F \rightarrow id \mid (E)$	{id,(}	{*,+,$,)}

Compiler Design

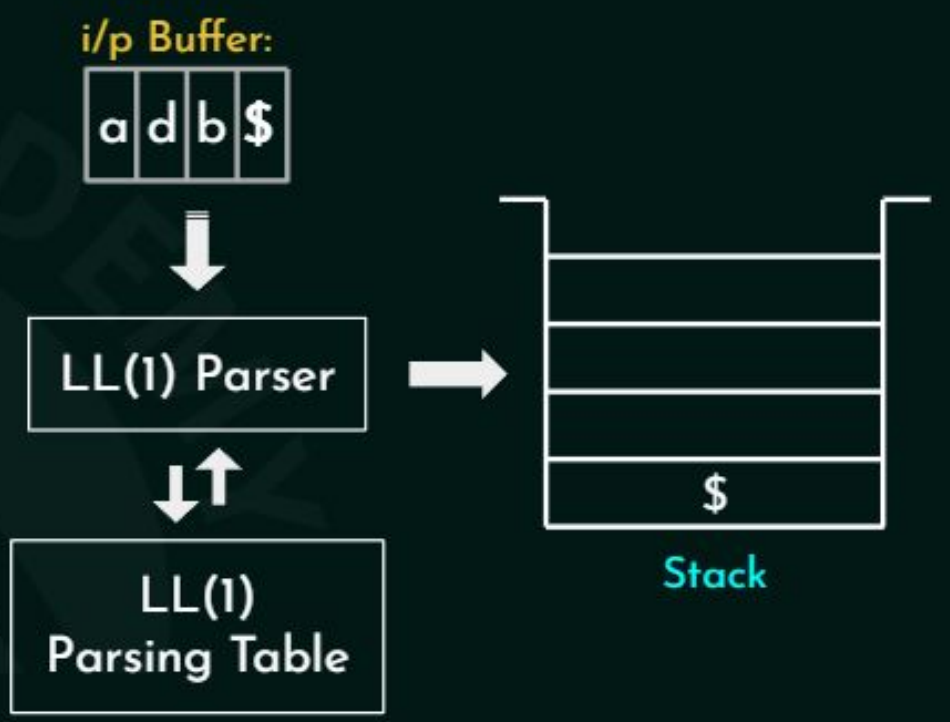
LL(1) Parsing

Outcome

- ☆ Illustration of LL(1) parsing procedure.

LL(1) Parsing:

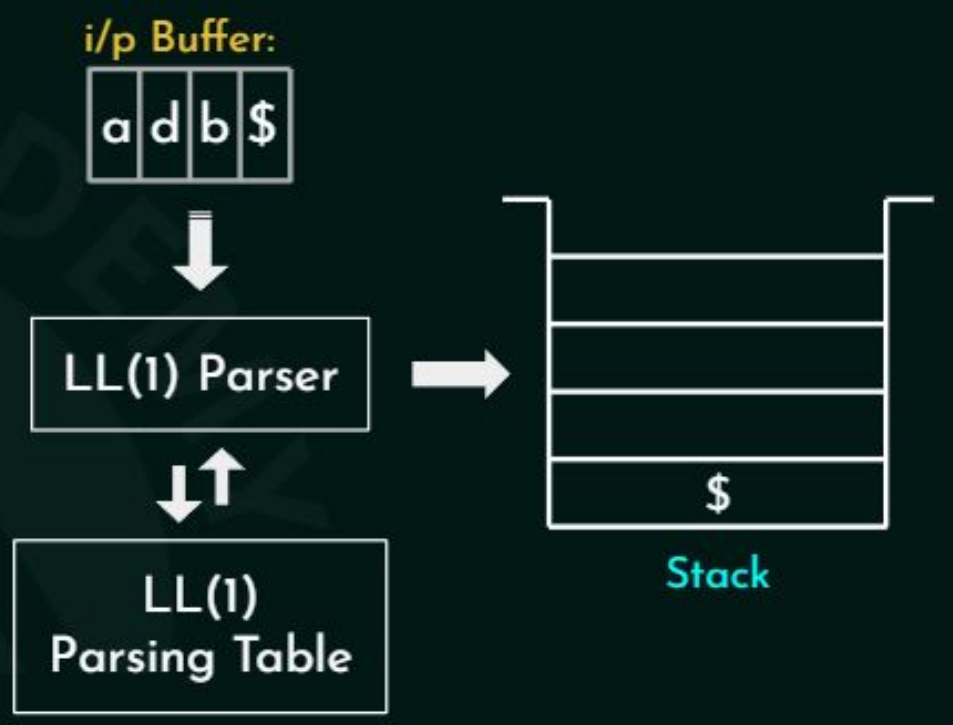
$S \rightarrow aABb$
$A \rightarrow c \mid \varepsilon$
$B \rightarrow d \mid \varepsilon$



LL(1) Parsing:

$S \rightarrow aABb$
$A \rightarrow c \mid \varepsilon$
$B \rightarrow d \mid \varepsilon$

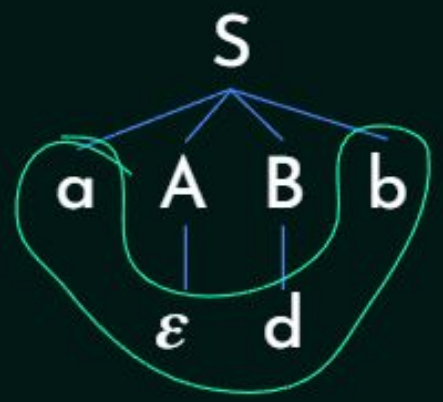
	FIRST	FOLLOW
$S \rightarrow aABb$	{a}	{$}
$A \rightarrow c \mid \varepsilon$	{c,ε}	{d,b}
$B \rightarrow d \mid \varepsilon$	{d,ε}	{b}

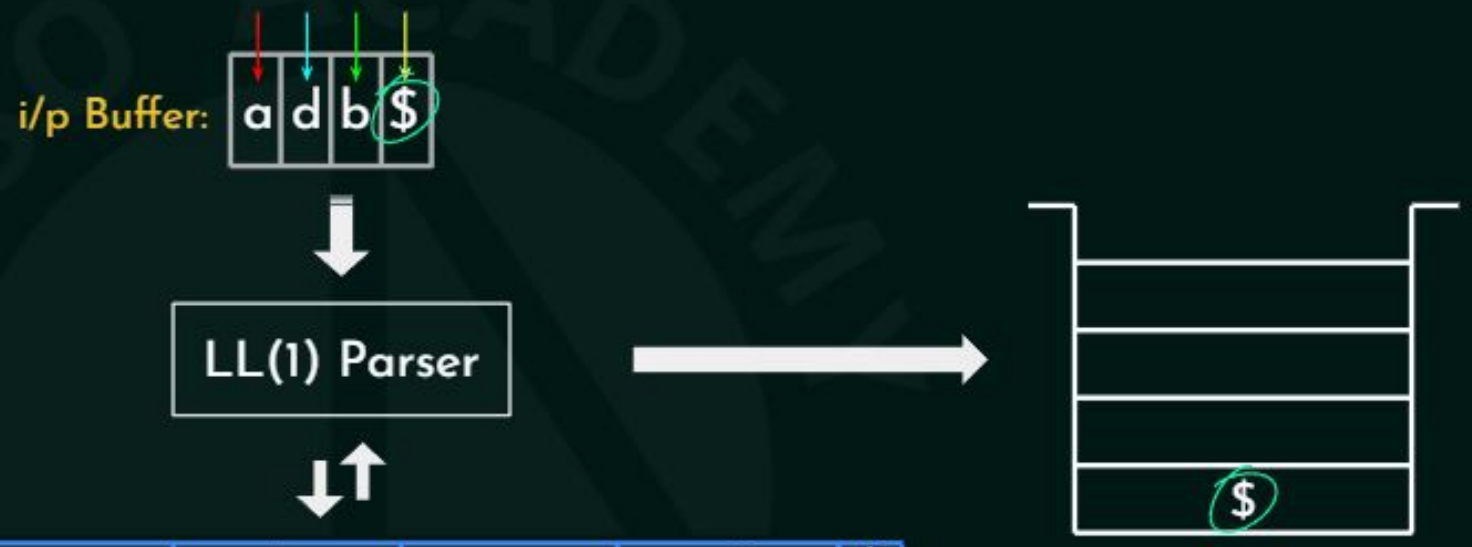


	a	b	c	d	$
S	$S \rightarrow aABb$				
A		$A \rightarrow \varepsilon$	$A \rightarrow c$	$A \rightarrow \varepsilon$	
B		$B \rightarrow \varepsilon$		$B \rightarrow d$	

LL(1) Parsing:

$S \rightarrow aABb$
$A \rightarrow c \mid \varepsilon$
$B \rightarrow d \mid \varepsilon$





	a	b	c	d	$
S	$S \rightarrow aABb$				
A		$A \rightarrow \varepsilon$	$A \rightarrow c$	$A \rightarrow \varepsilon$	
B		$B \rightarrow \varepsilon$		$B \rightarrow d$	

LL(1) Parsing Table

Compiler Design

LL(1) Parsing – Solved Problems (Set 1)

Outcome

- ☆ Five solved problems on whether the grammar is LL(1).

Q1: Find out whether the following grammar is LL(1):

$$S \rightarrow aSbS \mid bSaS \mid \varepsilon$$

Q1: Find out whether the following grammar is LL(1):

$$S \rightarrow aSbS \mid bSaS \mid \varepsilon$$

Sol. FIRST(S): {a, b, ε}

FOLLOW(S): {$, b, a}

Not LL(1) Grammar

	a	b	$
S	$S \rightarrow aSbS$ $S \rightarrow \varepsilon$	$S \rightarrow bSaS$ $S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Q2: Find out whether the following grammar is LL(1):

$$S \rightarrow (S) \mid \varepsilon$$

Q2: Find out whether the following grammar is LL(1):

$$S \rightarrow (S) \mid \varepsilon$$

Sol. FIRST(S): {(, ε}

FOLLOW(S): {$,)}

LL(1) Grammar

	(	)	$
S	$S \rightarrow (S)$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Q3: Find out whether the following grammar is LL(1):

$$S \rightarrow AaAb \mid BbBa$$
$$A \rightarrow \varepsilon$$
$$B \rightarrow \varepsilon$$

Q3: Find out whether the following grammar is LL(1):

$$S \rightarrow AaAb \mid BbBa$$
$$A \rightarrow \varepsilon$$
$$B \rightarrow \varepsilon$$

LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow AaAb \mid BbBa$	{a, b}	{$}
$A \rightarrow \varepsilon$	$\{\varepsilon\}$	{a, b}
$B \rightarrow \varepsilon$	$\{\varepsilon\}$	{b, a}

	a	b	$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	
B	$B \rightarrow \varepsilon$	$B \rightarrow \varepsilon$	

Q4: Find out whether the following grammar is LL(1):

$$S \rightarrow A \mid a$$

$$A \rightarrow a$$

Q4: Find out whether the following grammar is LL(1):

$$S \rightarrow A \mid a$$
$$A \rightarrow a$$

Not LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow A \mid a$	{a}	{$}
$A \rightarrow a$	{a}	{$}

Q5: Find out whether the following grammar is LL(1):

$$S \rightarrow aB \mid \varepsilon$$
$$B \rightarrow bC \mid \varepsilon$$
$$C \rightarrow cS \mid \varepsilon$$

Q5: Find out whether the following grammar is LL(1):

$$S \rightarrow aB \mid \varepsilon$$
$$B \rightarrow bC \mid \varepsilon$$
$$C \rightarrow cS \mid \varepsilon$$

LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow aB \mid \varepsilon$	$\{a, \varepsilon\}$	$\{\$\}$
$B \rightarrow bC \mid \varepsilon$	$\{b, \varepsilon\}$	$\{\$\}$
$C \rightarrow cS \mid \varepsilon$	$\{c, \varepsilon\}$	$\{\$\}$

	a	b	c	$
S	$S \rightarrow aB$			$S \rightarrow \varepsilon$
B		$B \rightarrow bC$		$B \rightarrow \varepsilon$
C			$C \rightarrow cS$	$C \rightarrow \varepsilon$

Compiler Design

LL(1) Parsing – Solved Problems (Set 2)

Outcome

- ☆ Four solved problems on whether the grammar is LL(1).

Q1: Find out whether the following grammar is LL(1):

$$S \rightarrow AB$$
$$A \rightarrow a \mid \varepsilon$$
$$B \rightarrow b \mid \varepsilon$$

Q1: Find out whether the following grammar is LL(1):

$$S \rightarrow AB$$
$$A \rightarrow a \mid \varepsilon$$
$$B \rightarrow b \mid \varepsilon$$

LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow AB$	$\{a, b, \varepsilon\}$	$\{\$\}$
$A \rightarrow a \mid \varepsilon$	$\{a, \varepsilon\}$	$\{b, \$\}$
$B \rightarrow b \mid \varepsilon$	$\{b, \varepsilon\}$	$\{\$\}$

	a	b	$
S	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$
A	$A \rightarrow a$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$
B		$B \rightarrow b$	$B \rightarrow \varepsilon$

Q2: Find out whether the following grammar is LL(1):

$$S \rightarrow aSA \mid \varepsilon$$

$$A \rightarrow c \mid \varepsilon$$

Q2: Find out whether the following grammar is LL(1):

$$S \rightarrow aSA \mid \varepsilon$$
$$A \rightarrow c \mid \varepsilon$$

Not LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow aSA \mid \varepsilon$	$\{a, \varepsilon\}$	$\{\$, c\}$
$A \rightarrow c \mid \varepsilon$	$\{c, \varepsilon\}$	$\{\$, c\}$

	a	c	$
S	$S \rightarrow aSA$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$
A		$A \rightarrow c$ $A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$

Q3: Find out whether the following grammar is LL(1):

$$S \rightarrow A$$
$$A \rightarrow Bb \mid Cd$$
$$B \rightarrow aB \mid \varepsilon$$
$$C \rightarrow cC \mid \varepsilon$$

Q3: Find out whether the following grammar is LL(1):

$$S \rightarrow A$$
$$A \rightarrow Bb \mid Cd$$
$$B \rightarrow aB \mid \varepsilon$$
$$C \rightarrow cC \mid \varepsilon$$

Sol.

	FIRST	FOLLOW
$S \rightarrow A$	{a, b, c, d}	{\$}
$A \rightarrow Bb \mid Cd$	{a, b, c, d}	{\$}
$B \rightarrow aB \mid \varepsilon$	$\{a, \varepsilon\}$	{b}
$C \rightarrow cC \mid \varepsilon$	$\{c, \varepsilon\}$	{d}

Q3: Find out whether the following grammar is LL(1):

Sol.

	FIRST	FOLLOW
$S \rightarrow A$	{a, b, c, d}	{\$}
$A \rightarrow Bb \mid Cd$	{a, b, c, d}	{\$}
$B \rightarrow aB \mid \varepsilon$	{a, ε}	{b}
$C \rightarrow cC \mid \varepsilon$	{c, ε}	{d}

	a	b	c	d	\$
S	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	
A	$A \rightarrow Bb$	$A \rightarrow Bb$	$A \rightarrow Cd$	$A \rightarrow Cd$	
B	$B \rightarrow aB$	$B \rightarrow \varepsilon$			
C			$C \rightarrow cC$	$C \rightarrow \varepsilon$	

Q3: Find out whether the following grammar is LL(1):

$$S \rightarrow A$$
$$A \rightarrow Bb \mid Cd$$
$$B \rightarrow aB \mid \varepsilon$$
$$C \rightarrow cC \mid \varepsilon$$

LL(1) Grammar

Sol.

	a	b	c	d	$
S	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	
A	$A \rightarrow Bb$	$A \rightarrow Bb$	$A \rightarrow Cd$	$A \rightarrow Cd$	
B	$B \rightarrow aB$	$B \rightarrow \varepsilon$			
C			$C \rightarrow cC$	$C \rightarrow \varepsilon$	

Q4: Find out whether the following grammar is LL(1):

$$S \rightarrow aAa \mid \varepsilon$$
$$A \rightarrow abS \mid \varepsilon$$

Q4: Find out whether the following grammar is LL(1):

$$S \rightarrow aAa \mid \varepsilon$$
$$A \rightarrow abS \mid \varepsilon$$

Not LL(1) Grammar

Sol.

	FIRST	FOLLOW
$S \rightarrow aAa \mid \varepsilon$	$\{a, \varepsilon\}$	$\{\$, a\}$
$A \rightarrow abS \mid \varepsilon$	$\{a, \varepsilon\}$	$\{a\}$

Home-work Problem:

Find out whether the following grammar is LL(1):

$$S \rightarrow iEtSS' \mid a$$
$$S' \rightarrow eS \mid \varepsilon$$
$$E \rightarrow b$$